

Of course. Now that you have a working dashboard, it's the perfect time to divide the work. Here are three distinct features you can assign to your teammates so they can work in parallel without conflicts.

1. Backend: Implement Duplicate File Detection

For: A team member who is comfortable with Python and backend logic.

- **The Goal:**
Make the "smart" part of the storage work. When a user uploads a file, the backend should check if an identical file already exists for that user.
- **The Work Involved:**
 1. **Modify the File Upload View:** They will work in the `backend/api/views.py` file, inside the `perform_create` method of the `FileListCreateView`.
 2. **Calculate the Hash:** Use Python's `hashlib` library to calculate the SHA-256 hash of the uploaded file.
 3. **Check the Database:** Before saving, query the database to see if another file owned by the current user already has the same hash.
 4. **Return a Response:**
 - If it's a **duplicate**, don't save the new file. The API should return an error (like a `409 Conflict` status) with a message like, "Error: This file already exists."
 - If it's **not a duplicate**, save the file and the new hash to the database as it currently does.

2. Frontend: Improve the File List UI

For: A team member who is focused on UI/UX and React.

- **The Goal:**
Make the file list on the dashboard look professional and interactive, with working download and delete buttons.
- **The Work Involved:**
 1. **Create a `FileItem` Component:** Build a new component (`FileItem.js`) that displays a single file's name and has two buttons: "Download" and "Delete."
 2. **Implement Delete Functionality:** The "Delete" button should show a confirmation (`window.confirm`) and then send a `DELETE` request to the backend API (`/api/files/<file_id>/`). After a successful deletion, it should refresh the file list.
 3. **Style the List:** Create a new CSS file (`FileItem.css`) to style the list, making it clean and user-friendly.
 4. **Integrate into Dashboard:** Update `FileDashboard.js` to use this new `FileItem` component for each file in the list.

3. Backend: Build Folder Management Logic

For: Another team member who can work on the backend.

- The Goal:
Add the ability for users to create and view folders to organize their files.
- The Work Involved:
 1. **Create a Folder Model:** In `backend/api/models.py`, create a new `Folder` model. It should have fields for `name`, `owner` (a foreign key to the `User`), and `parent_folder` (to allow for nested folders).
 2. **Update the File Model:** Add a foreign key to the `Folder` model in your existing `File` model, so each file knows which folder it belongs to.
 3. **Create Serializers:** In `backend/api/serializers.py`, create a new `FolderSerializer`.
 4. **Create API Views:** In `backend/api/views.py`, create new API views for creating a new folder and listing folders.
 5. **Update URLs:** Add new URLs in `backend/api/urls.py` for the folder-related actions.